

Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Радіотехнічний факультет
Кафедра прикладної радіоелектроніки

Звіт
про лабораторну роботу №3 за темою
«Злам шифру Цезаря частотним аналізом»
з дисципліни
«Захист даних»

Викладач:

Навроцький Денис Олександрович

Виконав:

Студент групи РС-61мн

Гандзюк Дмитро Васильович

Київ – 2026 р.

Лабораторна робота №3

Злам шифру Цезаря частотним аналізом

Мета роботи: на прикладі шифру Цезаря з'ясувати, як статистичні властивості природної мови дозволяють прочитати шифротекст без знання ключа; реалізувати такий злам, підтвердити його коректність на офіційних валідаторах платформи CodinGame та кількісно оцінити, скільки тексту потрібно різним вирішальним правилам для надійного визначення ключа.

Хід роботи

Постановка задачі

Задача взята з навчальної платформи CodinGame (розділ Training, рівень складності Medium) і має назву «Frequency-Based Decryption». На відміну від попередньої лабораторної роботи, тут не потрібно писати шифратор: на вхід подається вже зашифроване повідомлення, ключ не надається, і його треба відновити самостійно.

Формат даних задачі:

вхід : один рядок — повідомлення, зашифроване зсувом
вихід : один рядок — розшифроване повідомлення
обмеження: $67 \leq \text{довжина повідомлення} < 1000$

Умова накладає три додаткові вимоги. По-перше, зсув застосовується лише до літер: цифри, розділові знаки й пробіли залишаються на своїх місцях. По-друге, регістр літер має зберігатися. По-третє, еталонні частоти літер англійської мови задано безпосередньо в умові, і саме їх належить використовувати як еталон.

Приклад із умови:

вхід : Khoor Zruog ! Frqjudwxodwlrqv, brx kdyh vxffhvvixooob ...
вихід: Hello World ! Congratulations, you have successfully ...
ключ, знайдений частотним аналізом: $k = 3$

Окремо варто зробити термінологічне уточнення. В умові лабораторної роботи зсув названо перестановкою. У строгій класифікації шифрів циклічний зсув є *підстановкою* символів: він замінює кожну літеру іншою, але не змінює їхніх позицій у повідомленні; перестановний шифр, навпаки, переставляє самі символи (Handbook of Applied Cryptography, § 1.5.2). Надалі

для уникнення неоднозначності вживається термін «зсув Цезаря».

Модель шифру та простір ключів

Позначимо літери числами від 0 до 25 і виконуватимемо всі операції за модулем 26. Шифрування та розшифрування записуються так:

$$\begin{aligned}c[i] &= (m[i] + k) \bmod 26 && \text{— лише для літер} \\m[i] &= (c[i] - k) \bmod 26\end{aligned}$$

Зсуви утворюють циклічну групу порядку 26: послідовне застосування зсувів a і b дорівнює одному зсуву $a + b$. Це перевірено окремим автоматичним тестом і має практичний наслідок — багаторазове шифрування цим шифром нічого не додає, бо зводиться до одного зсуву. Уся конфігурація шифру описується одним числом.

Таблиця 3.1 – Простір ключів шифру Цезаря

Величина	Значення
Кількість ключів	26
Ентропія ключа за рівноймовірного вибору, біт	4,70
Орієнтовна відстань єдиності ($h \approx 1,5$ біта/символ), символів	1,5

За оцінки ентропії англійського тексту $h \approx 1,5$ біта/символ надлишковість мови становить $D = \log_2 26 - h \approx 3,20$ біта/символ, тому орієнтовна відстань єдиності дорівнює $n_0 = H(K) / D \approx 4,70 / 3,20 \approx 1,47$ символу.

Відстань єдиності тут наведено як орієнтир у прийнятій моделі мови, а не як жорстку межу: вона не гарантує ні неможливості зламу коротших повідомлень, ні успіху конкретного алгоритму на довших. Наскільки ця оцінка відрізняється від практики, показано далі.

Головний наслідок такого розміру простору ключів полягає в тому, що **через малий простір ключів практичний злам шифру Цезаря зводиться до повного перебору 26 можливих зсувів і вибору найкращого кандидата за статистичним критерієм**. Перебрати всі 26 варіантів тривіально; уся складність зосереджена в тому, щоб автоматично вирішити, котрий із 26 кандидатів є осмисленим текстом. Саме різні вирішальні правила й порівнює ця робота.

Програмна реалізація

Розв'язок виконано мовою Python 3 і продубльовано мовою C# на платформі .NET 8 (незалежна реалізація для перехресної перевірки). Вихідний код самодостатнього розв'язку для платформи наведено в додатку А. Решта вихідних текстів — ядро шифру, модуль зламу, статистичні метрики, набір автоматичних тестів і реалізація мовою C# — доступні в репозиторії роботи, посилання на який наведено в списку використаних джерел.

Реалізовано чотири вирішальні правила. Критерій **хі-квадрат** порівнює гістограму кандидата з еталонною таблицею з умови задачі (менше значення — краще). **Логарифмічна правдоподібність за частотами літер** робить те саме в термінах правдоподібності. **Логарифмічна правдоподібність за квадриграмами** враховує ще й порядок літер; її статистику взято з корпусу, опис якого наведено нижче, у підрозділі про експериментальну частину. Нарешті, **найвне правило** «найчастіша літера шифротексту відповідає Е» взагалі не перебирає кандидатів і слугує базовою лінією для порівняння.

Дрібна, але показова оптимізація в надісланому розв'язку: замість того щоб розшифровувати весь текст для кожного з 26 кандидатів, достатньо один раз побудувати гістограму шифротексту й циклічно зсувати її. Кожен кандидат тоді коштує 26 операцій замість L , а повне розшифрування виконується один раз — уже з відомим ключем. Обчислювальна складність такого розв'язку становить $O(L + 26^2)$ за часом, тобто $O(L)$, оскільки 26 — константа. Додаткова пам'ять — $O(L)$: власне гістограма та пошук ключа потребують лише $O(26)$, решту займає буфер вихідного рядка, який у наведеній реалізації накопичується повністю перед виведенням.

Верифікація розв'язку

Правильність реалізації підтверджено на чотирьох незалежних рівнях: офіційними валідаторами платформи, збереженими в репозиторії офіційними тестами, набором властивісних тестів і перехресним порівнянням двох реалізацій різними мовами.

Розв'язок надіслано на перевірку платформі CodinGame: пройдено всі п'ять видимих тестів, а підсумкова оцінка прихованих валідаторів становить **100 %**. Надісланий код у вікні редактора наведено на рисунку 3.1, результат

прогону тестів — на рисунку 3.2, підсумкову оцінку — на рисунку 3.3.

```
1
2 import sys
3
4 # Reference letter frequencies, in percent, exactly as given in the puzzle.
5 FREQ = [8.08, 1.67, 3.18, 3.99, 12.56, 2.17, 1.88, 5.27, 7.24, 0.14, 0.63,
6         4.04, 2.68, 7.38, 7.47, 1.91, 0.09, 6.42, 6.59, 9.15, 2.79, 1.08,
7         1.89, 0.21, 1.65, 0.07]
8
9 M = 26
10
11
12 def main() -> None:
13     message = sys.stdin.readline().rstrip("\n")
14
15     # Letter histogram of the ciphertext, case-insensitive.
16     counts = [0] * M
17     total = 0
18     for ch in message:
19         if "A" <= ch <= "Z":
20             counts[ord(ch) - 65] += 1
21             total += 1
22         elif "a" <= ch <= "z":
23             counts[ord(ch) - 97] += 1
24             total += 1
25
26     # Score every possible shift; shifting the histogram is equivalent to
27     # decrypting the whole message, but costs 26 operations instead of 1.
28     best_key, best_chi = 0, None
29     for k in range(M):
30         chi = 0.0
31         for i in range(M):
32             expected = total * FREQ[i] / 100.0
33             observed = counts[(i + k) % M]
34             chi += (observed - expected) ** 2 / expected
35         if best_chi is None or chi < best_chi:
36             best_key, best_chi = k, chi
37
38     # Decrypt with the winning key, preserving case and non-letters.
39     out = []
40     for ch in message:
41         if "A" <= ch <= "Z":
42             out.append(chr((ord(ch) - 65 - best_key) % M + 65))
43         elif "a" <= ch <= "z":
44             out.append(chr((ord(ch) - 97 - best_key) % M + 97))
45         else:
```

Рисунок 3.1 – Надісланий розв’язок у вікні редактора CodinGame

Test cases		
01	Test 1	▶ PLAY TEST
02	Test 2	▶ PLAY TEST
03	Test 3	▶ PLAY TEST
04	Test 4	▶ PLAY TEST
05	Test 5	▶ PLAY TEST

Рисунок 3.2 – Усі п’ять тестів платформи позначені як пройдені

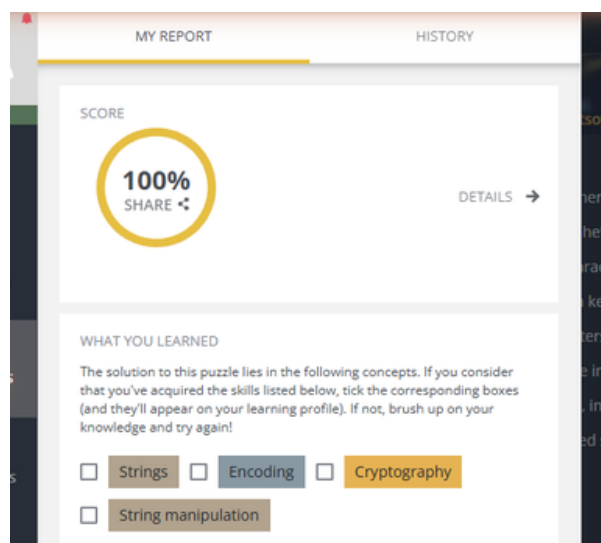


Рисунок 3.3 – Звіт платформи: оцінка валідаторів — 100 %

Входи та еталонні виходи всіх п'яти тестів збережено в репозиторії у файлі `tests/official_cases.json`. Окремий тест перевіряє самоузгодженість цього набору даних: записаний ключ має точно перетворювати очікуваний відкритий текст на збережений шифротекст, що виключає помилку переписування тестових даних. Перелік тестів наведено в таблиці 3.2.

Таблиця 3.2 – Офіційні тести CodinGame

№	Ключ	Довжина, символів	Результат
1	3	93	пройдено
2	8	227	пройдено
3	10	142	пройдено
4	14	119	пройдено
5	24	923	пройдено

У п'яти тестах використано п'ять різних ключів, тож вгадування одного зсуву тут не спрацювало б. П'ятий тест додатково перевіряє збереження типографського апострофа `U+2019` та інших нелітерних символів. Результат локального прогону:

```
$ python run.py selftest
Офіційні тести CodinGame
-----
[OK] Test 1  ключ 3 (істина 3)  Hello World ! Congratulations, ...
[OK] Test 2  ключ 8 (істина 8)  I rubbed shoulders with people ...
[OK] Test 3  ключ 10 (істина 10) You can fool all of the people ...
[OK] Test 4  ключ 14 (істина 14) Any code you've been writing ...
[OK] Test 5  ключ 24 (істина 24) Mr. and Mrs. Dursley, of number ...
-----
Пройдено 5 з 5
```

Окрім офіційних тестів, написано набір зі 107 автоматичних тестів. Вони перевіряють оборотність перетворення на 500 випадкових рядках; групову властивість зсувів; недоторканність нелітерних символів, зокрема цифр, розділових знаків і кирилиці; збереження регістру; інваріантність індексу відповідності щодо зсуву та, навпаки, чутливість критерію хі-квадрат; стійкість програми до входу без жодної літери й неминучу невдачу всіх правил на тексті з нетиповим розподілом літер. Реалізація мовою C# відновлює ті самі відкриті тексти, а окремий тест порівнює вихід обох реалізацій символ у символ. Увесь набір виконується автоматично при кожній зміні коду засобами GitHub Actions на трьох версіях Python.

Що зсув робить із частотами літер

Почнемо з того, чому частотна атака взагалі можлива. Зашифруємо англomовний текст обсягом 2497 символів зсувом $k = 3$ і порівняємо гістограми відкритого тексту й шифротексту з еталонною таблицею з умови задачі (рисунок 3.4).

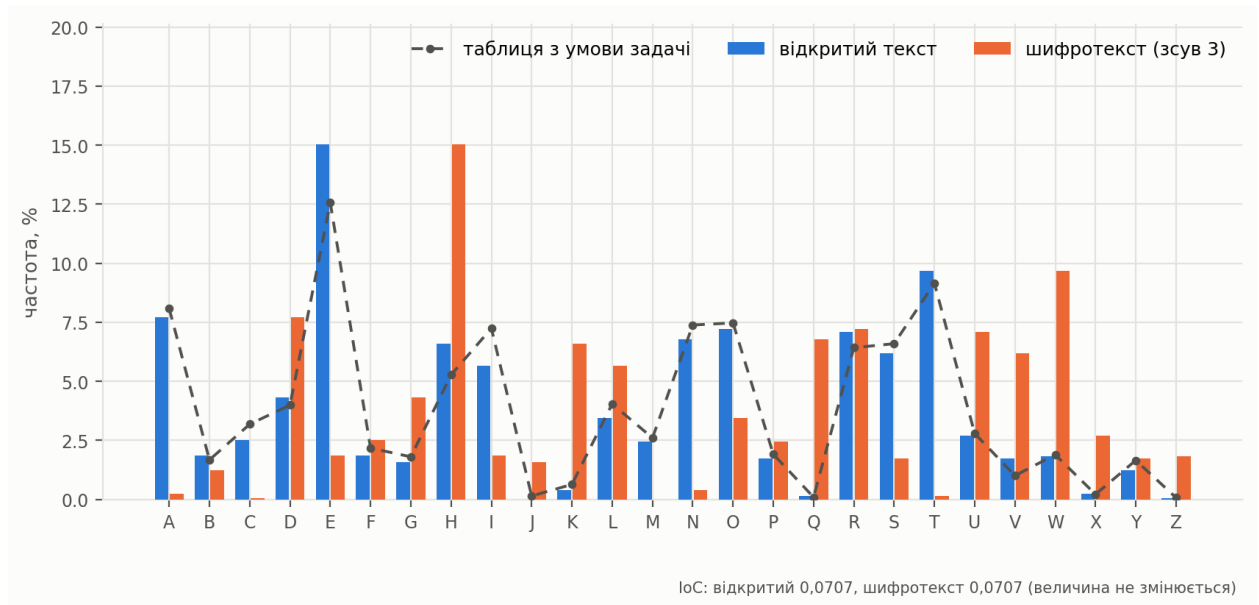


Рисунок 3.4 – Зсув не приховує статистику, а лише обертає гістограму

Результат принципово відрізняється від того, що спостерігалось в попередній лабораторній роботі. Там інкрементний зсув вирівнював частоти й робив розподіл майже рівномірним. Тут же фіксований зсув лише **обертає гістограму на k позицій**, не змінюючи її форми: найчастіша літера відкритого тексту Е переходить у літеру Н, тобто зсув піку точно дорівнює ключу. Кількісні показники наведено в таблиці 3.3.

Таблиця 3.3 – Статистика тексту до та після шифрування

Показник	Відкритий текст	Шифротекст
Найчастіша літера	Е	Н
Індекс відповідності	0,0707	0,0707
Хі-квадрат до таблиці з умови	54	15896
Емпірична ентропія окремих літер, біт	4,105	4,105

Індекс відповідності й ентропія окремих літер не змінилися зовсім — і це не випадковість, а точна властивість перестановки стовпчиків гістограми. Натомість хі-квадрат до еталонної таблиці зріс із 54 до 15896, бо гістограма

тепер зіставляється з еталоном «не в тій фазі». Саме ця відмінність між двома метриками й лежить в основі атаки.

Профіль хі-квадрат для 26 кандидатів

Перебираємо всі 26 можливих ключів і для кожного кандидата обчислюємо статистику хі-квадрат між його гістограмою та еталонною таблицею:

$$\chi^2(k) = \text{SUM по літерах } (\text{спостережено} - \text{очікувано})^2 / \text{очікувано}$$

Результат для повідомлення завдовжки 400 символів наведено на рисунку 3.5.

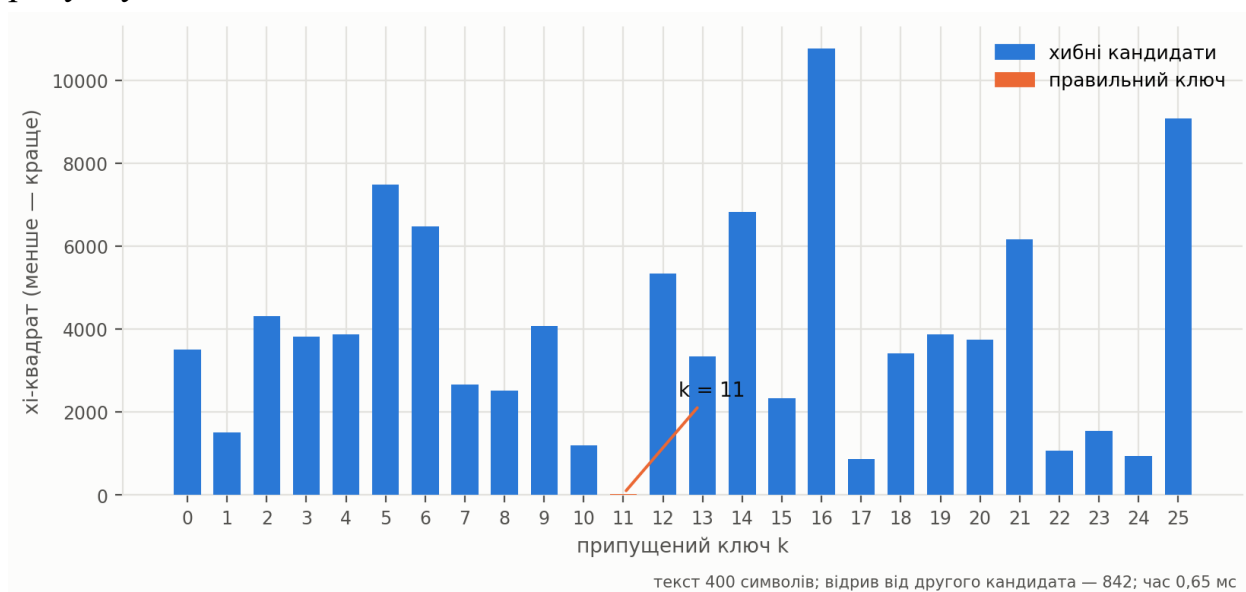


Рисунок 3.5 – Значення хі-квадрат для 26 кандидатів; правильний ключ дає різкий мінімум

Правильний ключ виділяється однозначно: його значення хі-квадрат у 37 разів менше, ніж у наступного за якістю кандидата, а весь перебір триває 0,65 мілісекунди. Це типова картина для повідомлень такої довжини; поведінку на коротших текстах розглянуто далі.

Чому ключ дає хі-квадрат, а не індекс відповідності

Індекс відповідності — класичний інструмент криптоаналізу, і на перший погляд він мав би теж підійти. Проте ці дві метрики поведуться протилежно, і різниця між ними варта окремого розгляду. Застосуємо до одного тексту всі 26 зсувів і подивимося, як змінюються обидві величини (рисунок 3.6).

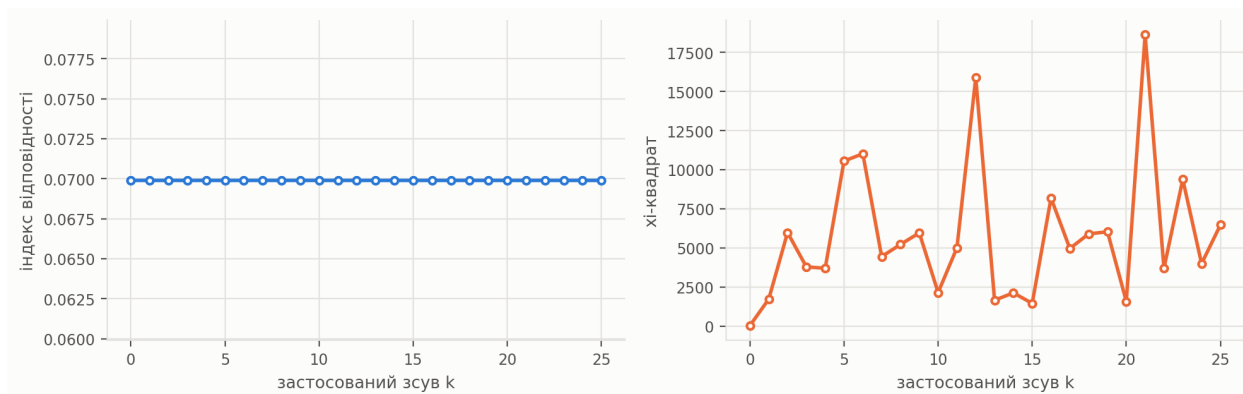


Рисунок 3.6 – Індекс відповідності не залежить від зсуву, а хі-квадрат залежить

Індекс відповідності не змінився взагалі: усі 26 значень збіглися до останнього знаку. Хі-квадрат натомість змінюється від 25 до 18642, тобто у 744 рази. Причина проста: зсув лише переставляє стовпчики гістограми, а індекс відповідності залежить тільки від їхнього набору, не від порядку. Хі-квадрат же порівнює гістограму з конкретним еталоном, і тому «відчуває» фазу.

Практичний висновок такий: індекс відповідності є інваріантним щодо зсуву Цезаря, тому не дає змоги визначити ключ. Водночас його значення може використовуватися як статистична характеристика розподілу символів і як допоміжний показник у криптоаналізі — зокрема, в останньому експерименті роботи його значення для шифротексту наближається до значення, характерного для рівномірного розподілу. У попередній лабораторній роботі індекс відповідності виявив поліалфавітну структуру шифру; тут усю роботу щодо ключа виконує хі-квадрат.

Скільки тексту потрібно різним правилам рішення

Це основний експеримент роботи. Для кожної довжини повідомлення бралось 200 випадкових фрагментів тексту, кожен шифрувався випадковим ключем, після чого всі чотири правила намагалися цей ключ відновити. Результати наведено на рисунку 3.7 і в таблиці 3.4.

Використані дані. Фрагменти нарізаються з файлу data/sample_plaintext.txt, що входить до репозиторію роботи: це суцільний англійський текст обсягом 2 501 байт (2497 символів після нормалізації пробілів), який містить лише літери A–Z, пробіли та розділові знаки; перед

нарізанням із нього прибрано розбиття на рядки. Статистику квадриграм узято з іншого, значно більшого корпусу: вісім книжок суспільного надбання з бібліотеки Project Gutenberg, зведених до послідовності літер A–Z після видалення службових текстів ліцензії, разом 5 623 458 літер; у модель збережено 4-грами з частотою не менше трьох, усього 54 626 записів; скрипт побудови моделі та сама модель зберігаються в репозиторії роботи. Зерна генераторів псевдовипадкових чисел зафіксовано (20260923 для цього експерименту, 4242 і 77 для двох наступних), тому весь прогін відтворюється число в число.

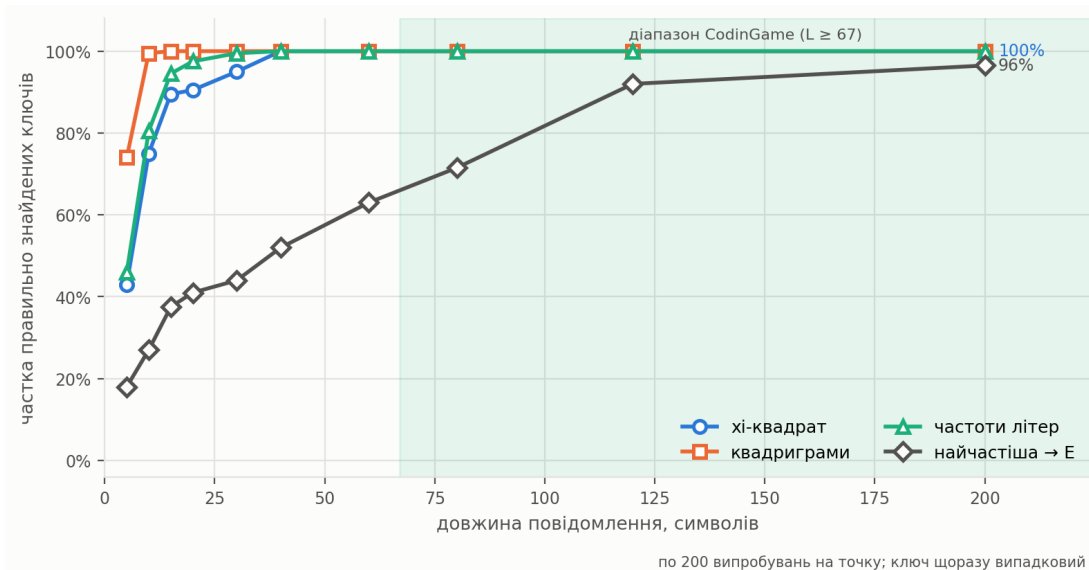


Рисунок 3.7 – Частка правильно знайдених ключів залежно від довжини повідомлення

Таблиця 3.4 – Частка правильно знайдених ключів, % (по 200 випробувань на точку)

Правило рішення	5	10	15	20	40	120	200
Квадриграми	74,0	99,5	100,0	100,0	100,0	100,0	100,0
Частоти літер	46,0	80,5	94,5	97,5	100,0	100,0	100,0
Хі-квадрат	43,0	75,0	89,5	90,5	100,0	100,0	100,0
Найчастіша → E	18,0	27,0	37,5	41,0	52,0	92,0	96,5

У цій серії випробувань квадриграми першими досягли 100 % — уже на довжині 15 символів; хі-квадрат і логарифмічна правдоподібність за частотами літер досягли 100 % на 40 символах, а найвне правило не досягло 100 % навіть на 200 символах (96,5 %). Перевага квадриграм на коротких

текстах закономірна: вони враховують не лише частоти літер, а й порядок, у якому ті стоять.

Зіставимо це з теоретичною оцінкою. Відстань єдиності для 26 ключів становить близько 1,5 символу. Це наближений орієнтир у прийнятій моделі мови, а не твердження про те, що ключ можна однозначно відновити за одним-двома символами: формула Шеннона має модельний характер і нічого не каже про обчислювальну процедуру. Практичні 15–40 символів — це розрив між інформаційно-теоретичною оцінкою та можливостями конкретного вирішального правила; наведені числа характеризують саме цю серію випробувань на використаному тексті.

Для умови задачі це означає таке. Мінімальна довжина повідомлення, передбачена умовою задачі, $L \geq 67$, суттєво перевищує довжини, на яких у проведеній серії випробувань усі три статистичні правила досягли 100 % правильних визначень ключа. Це робить вибір критерію хі-квадрат виправданим, але саме проходження валідаторів є окремим фактом і підтверджується не цією серією, а результатом платформи CodinGame, наведеним вище.

Довжина повідомлення сама по собі успіху не гарантує: вирішує не вона, а схожість розподілу літер на англійський. Контрприклад — рядок «I did it.», повторений вісім разів: 80 символів, тобто вдвічі більше за поріг із рисунка 3.7, але лише 48 літер і всього 3 різні з них. Жодне з чотирьох правил не знаходить справжнього ключа 3: хі-квадрат, частоти літер і квадриграми дають 18, наївне правило — 7. Тому твердження про 100 % стосується типових англійських текстів, а не будь-якого повідомлення достатньої довжини.

Чому помиляється наївне правило

Правило «найчастіша літера шифротексту відповідає E» цікаве тим, що помиляється не випадково, а передбачувано: його похибка безпосередньо пов'язана зі зсувом найчастішої літери *відкритого* тексту відносно E. Розподіл похибки на повідомленнях по 40 символів (2000 випробувань) наведено на рисунку 3.8.

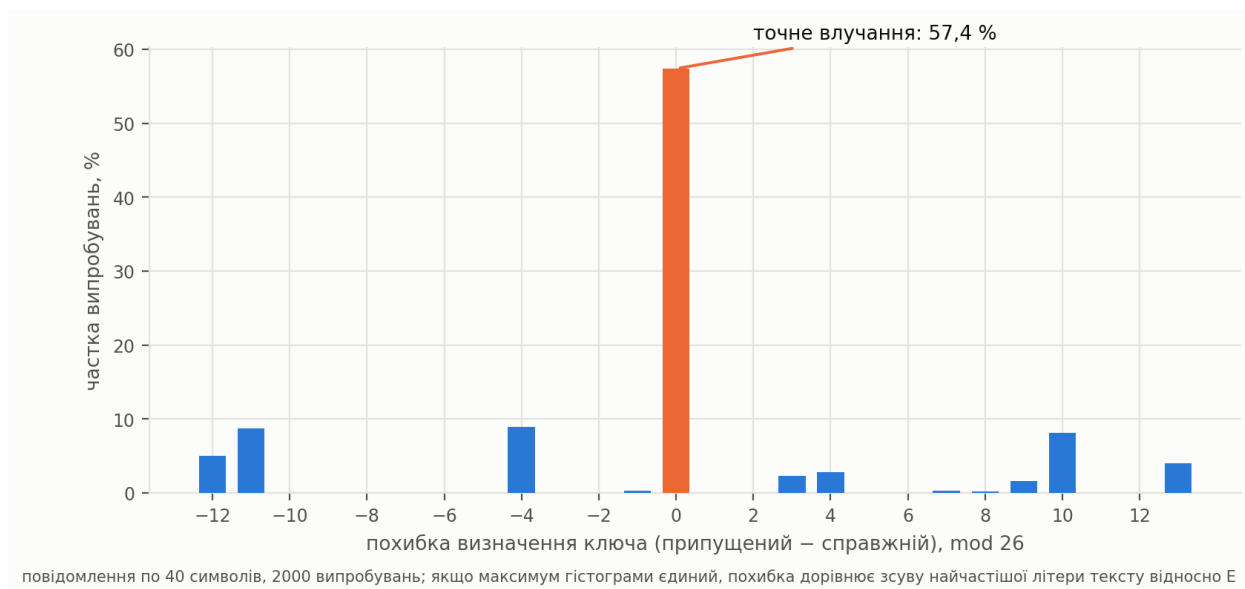


Рисунок 3.8 – Розподіл похибки наївного правила визначення ключа

Таблиця 3.5 – Найчастіша літера у фрагментах відкритого тексту

Літера	Е	А	О	решта
Частка випробувань, %	61,4	12,3	6,6	19,7

Точність правила становила 57,4 %, тоді як частка фрагментів, у яких найчастішою літерою справді виявилася Е, дорівнює 61,4 %. Розрив між цими величинами пояснюється нічиями за максимумом гістограми та правилом їх розв’язання. На фрагментах такої довжини лічильники малі, тому в 23,4 % випробувань максимум досягався одразу на кількох літерах; правило розв’язує нічию на користь літери з меншим номером в алфавіті, а циклічний зсув такого вибору не зберігає. Кількісно це показано в таблиці 3.6.

Таблиця 3.6 – Вплив нічий за максимумом на похибку наївного правила

Випадок	Частка випробувань	Виконання формули похибки	Точність правила
Максимум єдиний	76,65 % (1533)	1533 із 1533	65,1 %
Є нічия за максимумом	23,35 % (467)	не гарантовано	32,1 %

Отже, формула «похибка дорівнює зсуву найчастішої літери тексту відносно Е» є точною за умови, що максимум гістограми єдиний: у цій серії вона виконалася в усіх 1533 таких випробуваннях без жодного винятку. Методичний висновок від цього не змінюється: правило спирається рівно на одне спостереження, тоді як хі-квадрат використовує всі 26 і тому стійкіший

до коливань коротких вибірок.

Коли частотна атака перестає працювати

У постановці лабораторної роботи є пряма вимога: шифрограми мають бути без статистичних закономірностей, притаманних звичайному тексту. Перевіримо її експериментально: зашифруємо один і той самий текст звичайним зсувом Цезаря та зсувом, що для кожної позиції обирається випадково й незалежно (рисунк 3.9).

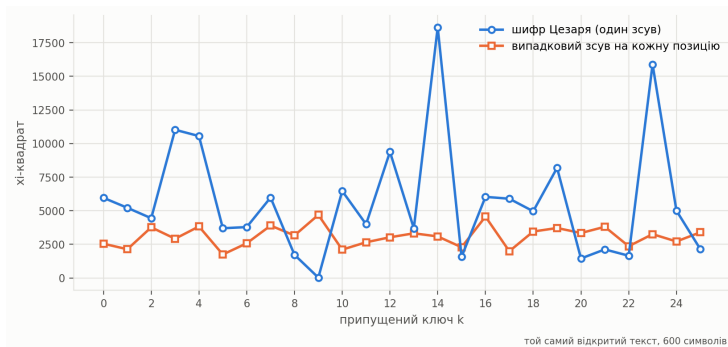


Рисунок 3.9 – Частотна атака працює лише тоді, коли шифр лишає статистичний слід

Таблиця 3.7 – Порівняння двох способів шифрування одного тексту

Шифр	Мін. хі-квадрат	Макс. хі-квадрат	Контраст	ІоС шифротексту
Цезар (один зсув)	25	18642	744×	0,0699
Випадковий зсув на кожну позицію	1743	4690	2,7×	0,0392

У другому випадку профіль хі-квадрат майже плаский, а індекс відповідності падає до 0,0392 — значення, характерного для рівномірного розподілу. Незалежні випадкові зсуви приховують саме частотний розподіл літер, тому розглянута атака на фіксований зсув стає непридатною; до того ж спільного ключа, який можна було б знайти перебором 26 варіантів, у такому шифрі немає. Це не означає, що жодних статистичних закономірностей не лишилося: пробіли, довжини слів, пунктуація та регістр залишаються видимими, бо шифр їх не торкається. Практичною схемою він теж не є — ключова послідовність завдовжки з повідомлення сама потребує безпечної передачі. Демонстрація пояснює лише те, на чому тримається частотний аналіз.

Висновки

1. Реалізовано злам шифру Цезаря без знання ключа: на CodinGame пройдено п'ять видимих тестів, оцінка прихованих валідаторів — 100 %. Реалізацію продубльовано мовами Python і C# та покрито 107 тестами.

2. Через малий простір ключів практичний злам шифру Цезаря зводиться до повного перебору 26 можливих зсувів і вибору найкращого кандидата за статистичним критерієм. Простір ключів відповідає ентропії 4,70 біта, тому вся складність — у вирішальному правилі.

3. Зсув зберігає форму частотного розподілу, лише обертаючи гістограму на k позицій. Індекс відповідності при цьому не змінюється зовсім — усі 26 значень збігаються до останнього знаку, — а χ^2 -квадрат змінюється у 744 рази. Тому ключ дає саме χ^2 -квадрат: інваріантний індекс нічого не каже про зсув.

4. Вирішальні правила різняться за вимогами до обсягу тексту: у цій серії квадриграми досягли 100 % на 15 символах, χ^2 -квадрат і частоти літер — на 40, а наївне правило — і на 200 ні. Водночас довжина успіху не гарантує: на тексті з 80 символів лише з трьома різними літерами помилилися всі правила.

5. Похибка наївного правила регулярна: якщо максимум гістограми єдиний, вона дорівнює зсуву найчастішої літери тексту відносно Е (без винятків на 1533 випробуваннях). Тому точність правила (57,4 % на фрагментах по 40 символів) близька до частки фрагментів із найчастішою Е (61,4 %); розрив дають нічий за максимумом, що трапилися у 23,4 % випробувань.

6. Вимогу умови щодо статистичних закономірностей перевірено вимірюванням: для шифру Цезаря значення χ^2 -квадрат у найкращого й найгіршого кандидатів відрізнялися приблизно у 744 рази, а для шифру з випадковим зсувом на кожен позицію — лише у 2,7 рази. Такі зсуви приховують частотний розподіл літер, через що розглянута атака стає непридатною, хоча пробіли, пунктуація та регістр зберігаються.

7. Практичний урок: для розглянутих класичних шифрів важливим чинником стійкості до частотного аналізу є те, наскільки шифротекст зберігає статистичні закономірності відкритого тексту.

Список використаних джерел

1. CodinGame. Frequency-Based Decryption [Електронний ресурс]. – Режим доступу:
<https://www.codingame.com/training/medium/frequency-based-decryption>
2. Caesar cipher [Електронний ресурс] // Wikipedia. – Режим доступу:
https://en.wikipedia.org/wiki/Caesar_cipher
3. Frequency analysis [Електронний ресурс] // Wikipedia. – Режим доступу:
https://en.wikipedia.org/wiki/Frequency_analysis
4. Index of coincidence [Електронний ресурс] // Wikipedia. – Режим доступу:
https://en.wikipedia.org/wiki/Index_of_coincidence
5. Menezes A., van Oorschot P., Vanstone S. Handbook of Applied Cryptography, § 1.5. – CRC Press, 1996. – Режим доступу:
<https://cacr.uwaterloo.ca/hac/about/chap1.pdf>
6. Shannon C. E. Communication Theory of Secrecy Systems // Bell System Technical Journal. – 1949. – Vol. 28, No. 4. – P. 656–715.
7. Beker H., Piper F. Cipher Systems: The Protection of Communications. – London: Northwood Books, 1982. – 436 p.
8. Project Gutenberg [Електронний ресурс]. – Режим доступу:
<https://www.gutenberg.org/> (перелік восьми використаних книжок наведено у файлі tools/build_ngrams.py репозиторію роботи)
9. Вихідний код роботи [Електронний ресурс]. – Режим доступу:
<https://github.com/RE22GDV/LR3-Frequency>

Додаток А

Розв'язок задачі, надісланий на платформу CodinGame

Нижче наведено повний текст файла `solution/codingame_solution.py`. Його алгоритм надіслано на платформу CodinGame й оцінено валідаторами на 100 %. Від надісланого варіанта цей файл відрізняється двома доповненнями, що не впливають на результат: заголовним коментарем з умовою задачі та описом методу і перевіркою входу без жодної літери, на якому χ^2 -квадрат ділив би на нуль (умова задачі такого входу не передбачає, тож на валідаторах ця гілка недосяжна; у реалізації мовою C# відповідна перевірка була від початку). Решта вихідних текстів роботи — ядро шифру, модуль зламу з чотирма правилами рішення, статистичні метрики, набір автоматичних тестів, скрипти експериментів і незалежна реалізація мовою C# — не наводиться тут, щоб не переобтяжувати звіт, і доступна в репозиторії роботи за посиланням, наведеним у списку використаних джерел.

```
"""
CodinGame - "Frequency-Based Decryption" (Medium)
https://www.codingame.com/training/medium/frequency-based-decryption

Self-contained solution, submitted as-is.
This file is kept ASCII-only. The executable code is exactly what was
submitted and accepted; only this header comment is added here.
The Ukrainian write-up lives in README.md and in src/freqcrack/.

Task: an English text was encrypted with a Caesar shift. The key is NOT
given. Recover it by letter-frequency analysis and print the plaintext,
preserving letter case and leaving non-alphabetic characters untouched.

Method: the key space holds only 26 elements, so breaking the cipher in
practice comes down to trying every shift and picking the best candidate
by a statistical criterion. Every candidate is scored with the chi-squared
statistic against the reference frequency table supplied by the puzzle,
and the candidate with the smallest value wins.

    chi2(k) = sum over letters of (observed - expected)^2 / expected

The histogram is built once and then rotated, and the message itself is
decrypted once, so the cost is  $O(L + 26^2)$  time; as 26 is a constant,
that is  $O(L)$ . Extra memory is  $O(L)$ : the histogram and the key search
need only  $O(26)$ , the rest is the output buffer.
"""

import sys

# Reference letter frequencies, in percent, exactly as given in the puzzle.
FREQ = [8.08, 1.67, 3.18, 3.99, 12.56, 2.17, 1.80, 5.27, 7.24, 0.14, 0.63,
        4.04, 2.60, 7.38, 7.47, 1.91, 0.09, 6.42, 6.59, 9.15, 2.79, 1.00,
        1.89, 0.21, 1.65, 0.07]

M = 26
```

```

def main() -> None:
    message = sys.stdin.readline().rstrip("\n")

    # Letter histogram of the ciphertext, case-insensitive.
    counts = [0] * M
    total = 0
    for ch in message:
        if "A" <= ch <= "Z":
            counts[ord(ch) - 65] += 1
            total += 1
        elif "a" <= ch <= "z":
            counts[ord(ch) - 97] += 1
            total += 1

    # No letters at all: there is nothing to analyse, so echo the input back.
    # The puzzle guarantees English text, but chi2 would divide by zero here.
    if total == 0:
        print(message)
        return

    # Score every possible shift; shifting the histogram is equivalent to
    # decrypting the whole message, but costs 26 operations instead of L.
    best_key, best_chi = 0, None
    for k in range(M):
        chi = 0.0
        for i in range(M):
            expected = total * FREQ[i] / 100.0
            observed = counts[(i + k) % M]
            chi += (observed - expected) ** 2 / expected
        if best_chi is None or chi < best_chi:
            best_key, best_chi = k, chi

    # Decrypt with the winning key, preserving case and non-letters.
    out = []
    for ch in message:
        if "A" <= ch <= "Z":
            out.append(chr((ord(ch) - 65 - best_key) % M + 65))
        elif "a" <= ch <= "z":
            out.append(chr((ord(ch) - 97 - best_key) % M + 97))
        else:
            out.append(ch)

    print("".join(out))

if __name__ == "__main__":
    main()

```